

Databases, ORMs, and Data Modeling

SWE 4537: Server Programming Lecture 2.1

Asaduzzaman Herok
Department of CSE
Islamic University of Technology

19 November 2025

Agenda

- 1 Database Paradigms: SQL vs NoSQL
- 2 Object-Relational Mapping (ORM)
- 3 Implementing SQLModel
- 4 Handling Requests
- 5 Summary & Resources

The Persistence Layer

The Problem

Variables in Python (e.g., `users = []`) live in **RAM**. When you restart the server (or your code crashes), that data is gone forever.

The Solution: Databases.

- 👉 We need a permanent place to store data (Hard Drive/SSD).
- 👉 Two main schools of thought:
 - ① **SQL (Relational)**: Strict, organized, like a filing cabinet.
 - ② **NoSQL (Non-Relational)**: Flexible, fast, like a dump bin.

SQL: The Strict Organizer

Theory: E.F. Codd (1970)

Based on Set Theory and First-Order Predicate Logic. Data is represented as tuples (rows) grouped into relations (tables).

Analogy: The Excel Spreadsheet

- 👉 Imagine a spreadsheet named "Students".
- 👉 Columns: Name, ID, GPA.
- 👉 **Constraint:** You cannot just write a "Phone Number" in row 5 if the column doesn't exist for everyone.
- 👉 Every row must follow the exact same structure.

Key Traits:

- 👉 **Structured Schema**
- 👉 **Relations** (Tables linked by IDs)
- 👉 **Avoids Redundancy** (Data lives in one place)

ACID Properties

SQL databases prioritize validity and consistency.

- 👉 **A - Atomicity:** All operations in a transaction succeed, or none do. (All-or-nothing).
- 👉 **C - Consistency:** The database proceeds from one valid state to another valid state.
- 👉 **I - Isolation:** Concurrent transactions do not affect each other.
- 👉 **D - Durability:** Once committed, data is saved permanently, even in power loss.

ACID: The Bank Transaction Example

SQL is obsessed with **Safety**. Think of a Bank Transfer from Alice to Bob.

- 👉 **A - Atomicity (All or Nothing):** *If money leaves Alice but the server crashes before reaching Bob, the money must go back to Alice. It never vanishes.*
- 👉 **C - Consistency (Follow Rules):** *The database ensures Alice cannot have a negative balance if the rules say " $\text{Balance} \geq 0$ ".*
- 👉 **I - Isolation (Wait your turn):** *If Alice sends money to Bob and Charlie at the exact same millisecond, the database processes them one by one.*
- 👉 **D - Durability (Written in Stone):** *Once the screen says "Success", the data is safe even if the building loses power.*

NoSQL: The Flexible Model

Theory: The CAP Theorem Response

Emerged to handle "Big Data" and rapid iteration where strict consistency is less critical than availability and partition tolerance.

Key Characteristics:

- 👉 **Schemaless:** No predefined structure. Data is often JSON.
- 👉 **Denormalized:** Data is duplicated to optimize for read speeds (avoiding Joins).
- 👉 **BASE Model:** Basically Available, Soft state, Eventual consistency.

NoSQL: The Flexible Folder

Analogy: A Folder on your Desktop

- 👉 You create a folder "Project Info".
- 👉 Inside, you throw in a PDF, a text file, and an image.
- 👉 They don't need to match. One file has text, another has pixels.
- 👉 **NoSQL** works like this. You throw "Documents" (JSON) into a "Collection".

Key Traits:

- 👉 **Schemaless** (Dynamic)
- 👉 **Denormalized** (Speed over organization)
- 👉 **Horizontal Scaling**

Visualizing the Difference

SQL (Relational)

- 👉 Data is chopped up.
- 👉 **Users** Table
- 👉 **Orders** Table
- 👉 To see an order, you must **JOIN** them.

NoSQL (Document)

- 👉 Data is nested.
- 👉 One **Order** Document contains:
 - ▶ Customer Name
 - ▶ Address
 - ▶ List of Items
- 👉 Everything in one grab.

Scaling Theory

How do we handle more users?

Vertical Scaling (Scale Up)

- ➡ Adding more power (CPU, RAM) to a **single** server.
- ➡ **Limit:** Hardware limitations (Moore's Law limits).
- ➡ **SQL Preference:** Traditional SQL relies heavily on this.

Horizontal Scaling (Scale Out)

- ➡ Adding **more servers** to a cluster.
- ➡ **Limit:** Theoretically infinite.
- ➡ **NoSQL Preference:** Designed for Sharding (distributing data).

Scaling: The Wedding Cake Analogy

Your app is popular. How do you handle more users?

Vertical Scaling (SQL Preference)

The "Bigger Machine" Approach

- 👉 Like making a wedding cake taller.
- 👉 You add more RAM, more CPU to one server.
- 👉 **Limit:** Eventually, the cake tips over (Physics limits).

Horizontal Scaling (NoSQL Preference)

The "More Machines" Approach

- 👉 Like buying 100 cupcakes instead of one big cake.
- 👉 You add more cheap servers.
- 👉 **Limit:** Almost infinite.

Decision Matrix: SQL vs NoSQL

Context	SQL (PostgreSQL, MySQL)	NoSQL (MongoDB)
Data Type	Consistent, structured (Financial, Inventory)	Rapidly changing, unstructured (Social Feeds, Logs)
Priority	Validity (ACID)	Speed & Scale
Changes	Hard (Must alter table columns)	Easy (Just add new field)

The Problem: Speaking Different Languages

Python



Speaks "Objects"

```
user.name = "Alice"
```

Database



Speaks "Tables"

```
UPDATE users SET...
```

They cannot understand each other directly.

The Solution: ORM as a Translator

Object Relational Mapper

Think of the ORM as a **bilingual translator** standing between your code and the database.

👉 You speak Python to the ORM:

```
session.add(new_user)
```

👉 The ORM speaks SQL to the Database:

```
INSERT INTO users (name) VALUES ('Alice');
```

Benefit: You never have to learn the complex SQL dialects.

Concept: The Object-Relational Impedance Mismatch

A set of conceptual and technical difficulties when translating between RDBMS and OOP.

- ❶ **Granularity:** A class in code (e.g., 'Address') might just be columns in a 'User' table, not a separate table.
- ❷ **Inheritance:** SQL does not natively support inheritance. ORMs must simulate this (e.g., Single Table Inheritance).
- ❸ **Identity:** In Python, 'a == b' checks memory address. In SQL, identity is based on Primary Key.
- ❹ **Associations:** OOP uses object references. SQL uses foreign keys.

Why use an ORM?

- 1 **Productivity:** Reduces boilerplate code. No need to write manual SQL strings.
- 2 **Abstraction:** You think in "Business Logic" (Objects), not "Storage Logic" (Tables).
- 3 **Maintainability:** Model definitions are in one place (the Code).
- 4 **Database Agnostic:** Switch from SQLite to PostgreSQL by changing one line of configuration. The ORM handles the dialect differences.

Security: The Bouncer (SQL Injection)

The Danger of Raw SQL:

- 👉 Input: `name = "Robert'); DROP TABLE students; --"`
- 👉 Query: `"SELECT * FROM students WHERE name = '" + name + " '"`
- 👉 Result: The database reads the command and deletes the table!

The ORM Protection:

- 👉 The ORM treats input strictly as **text**, not commands.
- 👉 It "sanitizes" the input before sending it.
- 👉 It acts like a security guard checking IDs before letting data in.

The "Black Box" Problem

ORMs are not magic. They have trade-offs:

- 👉 **Performance Overhead:** There is a cost to translating objects to SQL.
- 👉 **N+1 Select Problem:** Naive code can trigger thousands of database queries.
 - ▶ *Example: Looping through 100 users and fetching their profile photo one by one.*
- 👉 **Complex Queries:** For extremely complex analytics, raw SQL is often cleaner and faster than ORM syntax.

SQLModel: Best of Both Worlds

In FastAPI, we have a unique problem. We need two things:

- 1 **Pydantic:** To check data coming from the user (Validation).
- 2 **SQLAlchemy:** To talk to the database (ORM).

The Old Way: define the same class twice (once for validation, once for DB).

The SQLAlchemy Way: Define one class that inherits from both.

One Class to rule them all.

Installation

We need the ORM and the Database Driver.

```
# Install SQLAlchemy (includes SQLAlchemy and Pydantic)  
pip install sqlalchemy  
  
# If using PostgreSQL  
pip install psycopg2-binary  
  
# If using SQLite (Default for dev)  
# No extra install needed, it's built into Python
```

Defining the Model

models.py

```
from typing import Optional
from sqlalchemy import Field, SQLAlchemy
# table=True -> This is for the Database
# SQLAlchemy -> This is for Pydantic Validation
class Hero(SQLAlchemy, table=True):
    # primary_key=True -> The unique ID for the DB
    id: Optional[int] = Field(default=None, primary_key=True)

    # index=True -> Organize this column for faster
    # searching
    name: str = Field(index=True)

    secret_name: str
    age: Optional[int] = Field(default=None)
```

Analogy: Engine vs Session

The Engine (Factory)

- 👉 Think of it as the **Power Plant**.
- 👉 It's heavy. It takes time to start up.
- 👉 You create it **once** when the app starts.
- 👉 It holds the actual connection to the DB file/server.

The Session (Worker)

- 👉 Think of it as a **Light Switch**.
- 👉 It's temporary. You flip it on/off often.
- 👉 You create one for **every request**.
- 👉 It borrows power from the Engine, does the work, and closes.

Setting up the Engine

database.py

```
from sqlalchemy import create_engine, SQLAlchemy

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:/// {sqlite_file_name}"

# The Engine is global,
# connect_args is specific to SQLite threading needs
engine = create_engine(sqlite_url, connect_args={"
    check_same_thread": False})

def create_db_and_tables():
    # This function creates the tables if they don't
    # exist
    SQLAlchemy.metadata.create_all(engine)
```

Concept: The Session

- 👉 The **Engine** stays open forever (global).
- 👉 The **Session** is temporary.
- 👉 It represents a single "conversation" or "transaction" with the database.
- 👉 **Lifecycle:**
 - 1 Request arrives → Open Session.
 - 2 Do operations (Read/Write).
 - 3 Commit (Save) or Rollback (Undo errors).
 - 4 Close Session → Return connection to Pool.

Getting a Session

How do we give each user their own "Light Switch" (Session)?

Dependency Injection: FastAPI will run this function automatically before the path operation.

```
def get_session():  
    with Session(engine) as session:  
        yield session  
        # The 'yield' keyword pauses here.  
        # The path operation runs.  
        # After the response is sent, the code resumes  
        here  
        # and the session closes automatically.
```

Under the Hood: Connection Pooling

Analogy: The Library Books

- 👉 Establishing a connection to a database is slow (like buying a book).
- 👉 Instead, the Engine keeps a "Pool" of open connections (like a Library).
- 👉 **Start Request:** You check out a book (Connection).
- 👉 **Use Database:** You read/write to it.
- 👉 **End Request:** You **return** the book to the shelf. You don't throw it away.
- 👉 **Benefit:** The next user can pick up the same book instantly.

The Create Flow

```
@app.post("/heroes/")
def create_hero(hero: Hero, session: Session = Depends(
    get_session)):
    session.add(hero)           # 1. Stage the changes (Not
                                # saved yet)
    session.commit()           # 2. Save to DB (The point
                                # of no return)
    session.refresh(hero)       # 3. Get the new ID
                                # generated by DB
    return hero
```

The Read Flow

```
from sqlalchemy import select

@app.get("/heroes/")
def read_heroes(session: Session = Depends(get_session)):
    # 1. Write the query in Python syntax
    statement = select(Hero).where(Hero.age > 20)

    # 2. Execute the query
    heroes = session.exec(statement).all()

    return heroes
```

The Update Flow

Updating is a 3-step process: Find, Change, Save.

```
@app.patch("/heroes/{hero_id}")
def update_hero(hero_id: int, hero_data: Hero, session:
    Session = Depends(get_session)):
    # 1. Find existing
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404)
    # 2. Update fields
    hero.name = hero_data.name
    # 3. Commit
    session.add(hero)
    session.commit()
    session.refresh(hero)
    return hero
```

Summary

❶ Paradigms:

- ▶ SQL = Organizer (Strict, Safe).
- ▶ NoSQL = Folder (Flexible, Scalable).

❷ ORM: The Translator that lets us write Python instead of SQL.

❸ SQLAlchemy: Combines Validation (Pydantic) and Database (SQLAlchemy).

❹ Performance: Connection Pooling acts like a library, reusing connections for speed.

Home Activity: 15 Minutes

Goal: Connect a Hello World app to a SQLite Database.

- 1 Install `sqlmodel`.
- 2 Create a `models.py` file and define a `Student` class.
- 3 Create a `database.py` file and setup the engine.
- 4 Create a POST endpoint to save a `Student`.
- 5 **Challenge:** Use "DB Browser for SQLite" to open the file and see your data.

Further Reading



SQLModel Documentation:

<https://sqlmodel.tiangolo.com/>



FastAPI SQL Guide:

<https://fastapi.tiangolo.com/tutorial/sql-databases/>



Visualizing SQL Joins:

<https://joins.spathon.com/>



The CAP Theorem:

https://en.wikipedia.org/wiki/CAP_theorem



PostgreSQL Official Docs:

<https://www.postgresql.org/docs/>

Questions?

Thank You!

Thank you for your time and attention.
Any questions or clarifications?

